

SwarmIQ: Autonomous Multi-Drone Swarm Simulator

Comprehensive Technical Summary & Academic Presentation Guide

Project: SwarmIQ (Autonomous Robotics Sim)	Fleet Capacity: 100+ Virtual Drones in Real-Time
Core Stack: Python, FastAPI, Three.js, React	Control Mode: 100% Autonomous (No Manual Pilot)
Tick Rate: 25 Hz Server Simulation / 60 FPS Viewport	GitHub: github.com/Meghs21/SWARM_IQ_DRONE

1. Problem Statement & Project Objective

The Problem: In real-world drone applications (search & rescue, defense, environmental mapping, disaster relief), managing large fleets using individual human pilots is impossible. Traditional multi-drone setups suffer from communication lag, single points of failure (if a central controller fails), and the inability to dynamically respond to sudden obstacles and moving hazards.

The Objective: Build **SwarmIQ** — a decentralized, software-based 3D simulation of an autonomous drone swarm where up to **100 virtual drones coordinate without any manual piloting**. The swarm self-organizes, elects leaders dynamically, navigates 3D obstacle courses via A*, maintains formations (V, Line, Grid, Circle), and actively prevents collisions in real-time.

Key Evaluation Highlights to Present:

- **Decentralized Intelligence:** Every drone executes local physics, separation, and obstacle evasion.
- **Hierarchical Planning:** Only the elected leader solves global 3D A*; followers maintain formation.
- **Fault Tolerance:** If the leader is shot down or drops below 20% battery, automatic election elects a new leader with zero mission interruption.
- **Real-Time 3D Rendering:** 100 drones at 60 FPS in Three.js via GPU hardware instancing.

2. System Architecture & Tech Stack

SwarmIQ uses a high-performance decoupled Client-Server architecture. The server runs the physics loop at 25 Hz and broadcasts JSON state snapshots over WebSockets to a Three.js 3D viewport.

Subsystem	Technologies Used	Key Responsibilities & Capabilities
Backend Engine	Python 3.11, FastAPI, Uvicorn, Asyncio	25 Hz non-blocking physics loop, REST control API, WebSocket server, state management.
Math & Physics	NumPy, SciPy	Vectorized O(N²) Boids forces, pairwise distance matrices, kinematic integration, clamping.
3D Frontend Viewport	Three.js, WebGL	GPU InstancedMesh rendering (100 drones in 1 draw call), OrbitControls, lighting, shadow maps.
Command HUD UI	React 18, Vite, Tailwind CSS, Lucide	Telemetry dashboard, formation switcher, color themes, fault injector, preset scenario cards.

3. The Six Core Autonomous Algorithms (Explain to Teacher)

Every drone in SwarmIQ operates as an independent agent calculating forces on every tick. The resultant motion is governed by continuous force synthesis: **F_total = F_boids + F_formation + F_navigation + F_obstacle(APF) + F_collision**

Algorithm	How It Works (Simple Concept)	Technical Formulation / Innovation
1. Reynolds Boids Flocking	Bio-inspired flocking: keeps drones moving as a cohesive group without clumping.	Vectorized O(N²) NumPy implementation: Separation (1/r²), Alignment (velocity matching), Cohesion (center of mass). Decoupled in formation flight so geometry is preserved.
2. 3D Global A* Path Planning	High-level 3D GPS: calculates the shortest collision-free flight path around obstacles.	3D Voxel grid with 26-connectivity and obstacle safety inflation (2.5m). Raycast string-pulling prunes redundant waypoints. Computed solely for the leader to save 95% CPU.
3. Artificial Potential Field (APF)	Invisible repulsive force fields push drones away from obstacles like matching magnetic poles.	Khatib APF formulation: $F_{rep} = k_{rep} * (1/d - 1/d_0) * (1/d^2)$. Features quadratic scaling with close-proximity surge up to 40 N, dynamic formation attenuation, and 0.8m physical hull deflection.
4. Geometric Formation Control	Rigid geometric slot assignment: arranges the swarm into V, Line, Grid, or Circle.	Persistent slot sorting modulo 2π prevents criss-crossing. Velocity feedforward matches leader velocity, enabling zero-lag millimeter-accurate slot tracking.
5. Dynamic Leader Election	Democratic fleet election: picks the best drone as captain; auto-replaces it on failure.	Multi-factor fitness function: Score = 35% Battery + 30% Distance + 20% Connectivity + 15% Health. Instant 1-tick failover when battery < 20% or on hardware failure.
6. Drone-to-Drone Collision Avoidance	Emergency close-range bubble: prevents drones from colliding with peer drones in the swarm.	Hyperbolic proximity repulsion under 2.0m ($F_{evade} = k * (1/d - 1/r_{safe})/d^2 * \Delta pos$). Vectorized pairwise distance matrix with 3-tier event tracking: Warnings (3m), Near-Misses (2m), Collisions (0.6m).

Deep Dive: How Artificial Potential Fields (APF) Work in SwarmIQ

- **The Principle:** The destination waypoint creates an attractive potential well pulling the drone forward ($F_{att} = -\nabla U_{att}$). Every obstacle (pillar, box, moving hazard) generates a repulsive potential hill pushing drones away ($F_{rep} = -\nabla U_{rep}$).
- **Overcoming the Classic APF Trapping Problem:** In naive APF, when drones are in tight formation, the formation spring force (18 N) overpowers the repulsive field (0.53 N), dragging drones into obstacles. In SwarmIQ, we introduced **Dynamic Formation Attenuation**: near obstacles, formation pull is scaled down so drones fluidly part around obstacles like water around a stone, and smoothly reform on the other side!

4. Fleet Size & Formation Flexibility Across All Modes

Interactive Customization Matrix: SwarmIQ gives full real-time control to the user during evaluation:

Control Feature	Supported Options	Behavior Across Scenarios / Modes
Fleet Size Selector	20, 50, or 100 Drones	Available across ALL 5 MODES and custom missions! You can switch between 20, 50, or 100 drones at any time. Scenario obstacles are preserved and drones re-slot dynamically.
Formation Selector	V, Line, Grid, Circle	Available in ALL 5 MODES ! You can switch formations at any moment — even while flying at full speed — and the drones smoothly morph in mid-air.
Follower Color Palette	Emerald, Violet, White, Amber, Cyan	Instantly updates follower drone materials and glow in the Three.js viewport without reloading.

Fault Injection	Kill Leader Button	Simulates sudden leader drone failure; demonstrates instant 40ms autonomous leader re-election.
-----------------	--------------------	---

5. Preset Demonstration Scenarios for Teacher Demo

Scenario	Default Fleet & Formation	What to Show the Teacher
1. Open Field	20 Drones • V-Wing (Can switch 20/50/100 & V/Line/Grid/Circle)	Clean baseline flight. Show formation keeping, then switch formation to Circle to watch smooth mid-air morphing with 0.007m circular precision.
2. Obstacle Course	50 Drones • V-Wing (Can switch 20/50/100 & V/Line/Grid/Circle)	3D A* waypoint path (green glowing line) avoiding static high-rise pillars. Drones part around pillars using Artificial Potential Fields (APF).
3. Dynamic Hazards	50 Drones • Line (Can switch 20/50/100 & V/Line/Grid/Circle)	Moving hazard spheres patrol the space. Drones dynamically deflect away using real-time Artificial Potential Fields.
4. Leader Failure	50 Drones • Grid (Can switch 20/50/100 & V/Line/Grid/Circle)	Click 'Fail Leader' button in HUD. The leader loses power, drops, and the next best drone is instantly elected captain without halting.
5. 100-Drone Swarm	100 Drones • Circle (Can switch 20/50/100 & V/Line/Grid/Circle)	Full-scale stress test. 100 drones forming a 55m ring. Demonstrates 60 FPS Three.js InstancedMesh performance.

6. Teacher Viva / Presentation Q&A; Cheat Sheet

Q1: Why didn't you compute 3D A* path planning for all 100 drones?

Answer: Computing 3D A* across a 3D voxel grid for 100 individual drones every frame would be computationally prohibitive ($O(N * \text{Voxel}^3)$). Instead, SwarmIQ uses a bio-inspired Leader-Follower hierarchy: only the leader computes the global A* path. Followers maintain geometric slots and use reactive local potential fields, reducing CPU load by over 95% while keeping the entire swarm safe.

Q2: How do you prevent drone-to-drone collisions inside the swarm?

Answer: We use a multi-tiered safety system: First, Boids Separation exerts a $1/r^2$ repulsion if drones get within 3.0m. Second, our Collision Avoidance engine calculates hyperbolic repulsive forces for pairs under 2.0m. Third, the persistent formation slot assignment guarantees that each drone has an assigned spatial slot separated by 3.5m.

Q3: How does the system handle leader failure?

Answer: Every tick, the Leader Election algorithm monitors the current leader's health and battery. If the leader's battery drops below 20% or a hardware failure occurs, a multi-factor election runs across all active followers. The drone with the highest score (combining battery level, distance to target, and mesh connectivity) is elected leader, and the A* path is replanned instantly.

Q4: How does Three.js render 100 drones at 60 FPS in a web browser?

Answer: Standard Three.js rendering creates 1 draw call per 3D mesh (which would lag at 100 drones). We used Three.js `InstancedMesh`. This allows all 100 follower drones to share a single geometry and material, rendering the entire swarm in exactly 1 GPU draw call, delivering a smooth 60 FPS even on standard laptops.

Q5: What is the green vertical pole and green line in the simulation?

Answer: The vertical green pillar with a spinning radar ring is the Destination Beacon (the mission objective coordinates). The glowing green line on the ground/air is the 3D A* Global Planned Path computed for the leader to avoid obstacles.

Q6: How does the Artificial Potential Field (APF) algorithm work and avoid getting drones trapped?

Answer: Each obstacle generates an invisible repulsive potential field ($F_{rep} = -\nabla U_{rep}$). Within 2.5m, an emergency surge ramps the repulsive thrust up to 40 N. To prevent the classic problem where formation holding force drags drones through obstacles, SwarmIQ uses 'Dynamic Formation Attenuation' — scaling down formation pull near obstacles so the swarm fluidly parts around obstacles like water around a stone, while a 0.8m physical hull buffer cancels any inward velocity.

7. Conclusion & Project Deliverables

SwarmIQ successfully fulfills all requirements of a real-time, software-based autonomous drone swarm simulator. All 37 automated test cases pass cleanly, verified across mathematical accuracy, dynamic failover, obstacle avoidance, and high-frequency WebSocket streaming. The complete source code is modular, well-documented, and hosted on GitHub at https://github.com/Meghs21/SWARM_IQ_DRONE.